Hyper-V vs. VMware: Overhead da Virtualização

Henrique Laia (M16491)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
henrique.laia@ubi.pt

Alexandre Santos (M16740)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
alexandre.minhoto.santos@ubi.pt

Daniel Basílio (E11929)
Departamento de Informática
Universidade da Beira Interior
Covilhã, Portugal
daniel.basilio@ubi.pt

***Resumo*—Este trabalho compara o desempenho de um hipervisor nativo (Hyper-V) e de um hipervisor baseado em anfitrião (VMware Workstation 17.6) no mesmo portátil, com máquinas virtuais Ubuntu Server 26.04 iguais. Cada teste foi repetido 10 vezes e comparado com o sistema nativo. No processador, os dois hipervisores perdem 2–11% face ao nativo. No acesso aleatório à memória, perdem 27% (Tipo 1) e 47% (Tipo 2). No disco, o Tipo 2 é mais rápido com um pedido de cada vez, por causa da *cache* do anfitrião, mas perde 88% de IOPS com 32 pedidos em fila. Na rede, perde 33–66% de débito.**

***Palavras-chave*—virtualização, Hyper-V, VMware Workstation, hipervisor Tipo 1, hipervisor Tipo 2, overhead, E/S**

I. Introdução

A virtualização é a base dos centros de dados e da computação em nuvem. Permite juntar vários sistemas isolados num só servidor físico, o que aproveita melhor o hardware e facilita a gestão [3], [4]. Quem torna isto possível é o hipervisor: fica entre o hardware e os sistemas convidados e controla o acesso ao processador, à memória e aos dispositivos de Entrada/Saída (E/S). Por isso, a forma como o hipervisor está construído influencia o desempenho das máquinas virtuais.

Os hipervisores dividem-se em dois tipos [4], [7]. O Tipo 1 (nativo ou *bare-metal*) corre diretamente sobre o hardware. O Tipo 2 (*host-based*) corre como uma aplicação dentro de um sistema operativo anfitrião. Costuma assumir-se que o Tipo 2 é sempre mais lento. No entanto, com a virtualização assistida por hardware, o código das máquinas virtuais já corre quase à velocidade nativa [9], [10], e o Tipo 2 pode até tirar partido de recursos do anfitrião, como a *cache* de ficheiros. Interessa, por isso, perceber em que subsistemas há *overhead* e em que situações aparece.

Neste trabalho, o Microsoft Hyper-V [1] e o VMware Workstation [2] foram instalados e configurados, em *single node*, no mesmo portátil, e mediu-se o desempenho do processador, da memória, do disco e da rede. Os principais contributos são: (i) uma comparação com recursos e CPU virtuais (vCPU) equivalentes, em que se confirmou o modo de execução de cada hipervisor e se testou a significância das diferenças; (ii) a explicação, com a lei de Little [14], do comportamento do disco, que favorece um ou outro hipervisor consoante o número de pedidos em fila; (iii) a medição do *overhead* de cada hipervisor face ao sistema sem virtualização; e (iv) um teste ao efeito do escalonador do Hyper-V.

O resto do artigo está organizado assim: a Secção II apresenta o enquadramento e os trabalhos relacionados; a Secção III, a arquitetura e os métodos; a Secção IV, a configuração experimental; a Secção V, os resultados; a Secção VI, as limitações; e a Secção VII, as conclusões.

Trabalho realizado no âmbito da Unidade Curricular de Virtualização e Centros de Dados, Departamento de Informática, Universidade da Beira Interior.

II. Background e Trabalhos Relacionados

A. Virtualização x86 Assistida por Hardware

Segundo Popek e Goldberg [7], uma arquitetura só é virtualizável se todas as instruções sensíveis forem privilegiadas, o que não acontecia no x86 original. Por isso, as primeiras soluções usavam tradução binária [9] ou paravirtualização, como o Xen [8]. Mais tarde, as extensões Intel VT-x e AMD-V criaram um modo de execução próprio para o hipervisor, e as tabelas de páginas de segundo nível (*Extended/Nested Page Tables*, EPT/NPT) passaram a fazer em hardware a tradução dos endereços do convidado [10]. Esta tradução tem um custo: quando há uma falha no TLB (*Translation Lookaside Buffer*), é preciso percorrer as tabelas de páginas do convidado e as do hipervisor. Com páginas de 4 KiB, isto pode exigir até 24 acessos à memória; com páginas grandes (2 MiB), são muito menos [10]. O outro custo importante são as saídas para o hipervisor (*VM exits*), sobretudo quando o convidado acede a dispositivos.

Os dispositivos virtuais podem ser de dois tipos. Os *emulados* imitam hardware real, como a placa de rede Intel `e1000` ou o controlador de disco LSI Logic, e cada acesso aos seus registos obriga a uma saída para o hipervisor. Os *paravirtualizados* usam controladores que sabem que estão numa máquina virtual e trocam vários pedidos de uma vez através de memória partilhada, o que é mais eficiente [8], [9].

B. Hipervisor Nativo: Microsoft Hyper-V

No Hyper-V, o hipervisor (`hvix64`/`hvax64`) arranca antes do Windows, que passa a correr como *partição raiz*. As máquinas virtuais correm em *partições filhas* e usam dispositivos paravirtualizados: os controladores do convidado (*Virtualization Service Clients*, VSC, como `hv_storvsc` e `hv_netvsc`) enviam os pedidos pelo barramento *VMBus* para os fornecedores de serviço (*Virtualization Service Providers*, VSP) da partição raiz, que os tratam em modo núcleo [1]. O Hyper-V é um hipervisor de Tipo 1 tanto no Windows Server

como no Windows 11. As edições diferem nas funcionalidades de centro de dados (como o *Failover Clustering* e o Hyper-V Replica) e no escalonador usado por omissão: *root* nas edições cliente, *classic* no Windows Server 2016 e *core* a partir do 2019 [11]. No *root scheduler*, é o Windows que escalona os vCPU, como se fossem *threads*. No *core scheduler*, o hipervisor atribui núcleos físicos inteiros a cada máquina e nunca divide os *threads* SMT (*Simultaneous Multithreading*) de um núcleo entre máquinas diferentes, por razões de segurança [11].

C. Hipervisor Baseado em Anfitrião: VMware Workstation

O VMware Workstation instala um monitor de máquinas virtuais (*Virtual Machine Monitor*, VMM) e um controlador no núcleo do Windows. No entanto, a emulação dos dispositivos e o acesso ao disco virtual (`.vmdk`) são feitos pelo processo `vmware-vmx`, em modo utilizador [2], [9]. Por isso, a E/S do convidado chega ao NTFS através do Windows e pode ser servida pela *cache* de ficheiros do anfitrião. Também a memória do convidado pertence ao processo `vmware-vmx` e é gerida pelo Windows. Quando o hipervisor do Windows está ativo (por exemplo, com o Hyper-V, a segurança baseada em virtualização ou o WSL2, *Windows Subsystem for Linux*), o VMware não consegue usar o seu monitor nativo (modo `CPL0`, nível de privilégio 0) e passa a correr sobre a *Windows Hypervisor Platform* (modo `ULM`, *User-Level Monitor*) [12]. Nesse caso, deixa de ser um Tipo 2 "puro".

D. Trabalhos Relacionados

Adams e Agesen [9] mostraram que as primeiras versões do VT-x eram, em algumas cargas, mais lentas do que a tradução binária, o que mudou com as EPT/NPT [10]. Barham *et al.* [8] mediram o custo da paravirtualização no Xen, e Felter *et al.* [13] concluíram que, no KVM (*Kernel-based Virtual Machine*), o *overhead* do processador e da memória é desprezável e que o maior custo está na E/S. Na Universidade da Beira Interior, Louro [3] comparou uma infraestrutura de elevada disponibilidade com Hyper-V e outra com contentores Docker. Usou o IOzone (sistema de ficheiros), o RAMspeed (memória), o iPerf3 (rede) e o Geekbench (processador), em três ambientes por infraestrutura (nativo, virtualizado e aninhado), mas não indicou o número de repetições nem a dispersão dos resultados. Este trabalho avalia os mesmos subsistemas com ferramentas abertas e configuráveis, cujos resultados têm unidades físicas: o `sysbench` [5] para o processador e a memória, o `fio` [6] para o disco, com E/S direta e várias profundidades de fila, e o `iperf3` para a rede. Acrescenta ainda repetições e análise estatística.

III. Arquitetura e Métodos

A. Arquitetura dos Cenários

A Fig. 1 mostra as duas pilhas de software. No Tipo 1, os pedidos de disco e de rede do convidado passam por controladores paravirtualizados até à partição raiz, onde são tratados em modo núcleo. No Tipo 2, os pedidos chegam a dispositivos emulados (LSI Logic e `e1000`, a configuração predefinida), e o processo `vmware-vmx` transforma-os em operações sobre ficheiros e sobre o adaptador virtual `VMnet8` do anfitrião.

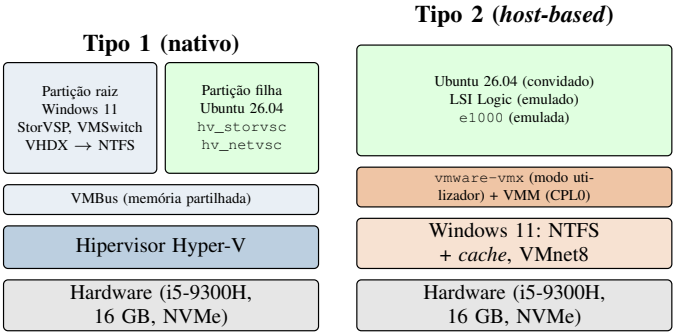


Figura 1. Pilhas de software dos dois cenários e respetivos dispositivos virtuais.

B. Formulação Estatística

Cada métrica foi medida $N = 10$ vezes. Para cada uma, calcula-se a média μ, o desvio-padrão amostral

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu)^2}, \tag{1}$$

e o intervalo de confiança a 95%, $\mu \pm t_{0{,}975;9}\,\sigma/\sqrt{N}$, com $t_{0{,}975;9} = 2{,}262$. Para saber se a diferença entre dois cenários é significativa, usa-se o teste t de Welch, que não exige variâncias iguais, com o limite $p < 0{,}05$.

C. Quantificação do Overhead

Tomando o Hyper-V como referência, a variação do Tipo 2 é

$$\Delta\% = \frac{\mu_{\text{Tipo 2}} - \mu_{\text{Tipo 1}}}{\mu_{\text{Tipo 1}}} \times 100. \tag{2}$$

Nas métricas de débito (eventos/s, MiB/s, IOPS, ou operações de E/S por segundo, Mbit/s), um $\Delta\%$ negativo significa que o Tipo 2 é pior; nas de latência, o Tipo 2 é pior quando $\Delta\%$ é positivo.

D. Modelo de Filas para o Armazenamento

Segundo a lei de Little [14], quando há Q pedidos em curso e a latência média é $\bar{L}$, o débito é

$$\text{IOPS} = \frac{Q}{\bar{L}}. \tag{3}$$

Se o caminho de E/S consegue tratar vários pedidos em paralelo, aumentar Q aumenta os IOPS sem aumentar muito a latência. Se só trata um pedido de cada vez, os IOPS deixam de subir e a latência cresce na mesma proporção que Q. Esta relação permite distinguir um caminho *lento* de um caminho *serializado*.

Tabela I
SISTEMA ANFITRIÃO

Componente	Especificação
Processador	Intel Core i5-9300H, 4 núcleos / 8 *threads*, 2,4 GHz
Memória	16 GB DDR4-2666 (2 × 8 GB)
Armazenamento	SSD NVMe Samsung PM981 (MZVLB1T0), 1 TB
Sistema operativo	Windows 11 Home 25H2, *build* 26200
Energia	Plano "Elevado desempenho", ligado à corrente

Tabela II
CONFIGURAÇÃO DAS MÁQUINAS VIRTUAIS

Parâmetro	Hyper-V (Tipo 1)	VMware (Tipo 2)
Versão	config. VM 12.0, Geração 2	Workstation 17.6.4, HW v21
Convidado	Ubuntu Server 26.04.1 LTS, *kernel* 7.0.0-38	
vCPU	2, sem SMT (1 *socket* × 2)	2, sem SMT (2 *sockets* × 1)
Memória	4096 MB (fixa)	4096 MB
Disco (25 GB)	`.vhdx` dinâmico	`.vmdk` esparso
Controlador de disco	SCSI sintético (VMBus)	LSI Logic (emulado)
Escalonador E/S conv.	`none`	`mq-deadline`
Rede	`hv_netvsc`, *Default Switch*	`e1000`, NAT (VMnet8)
Sistema de ficheiros	`ext4` sobre LVM	
Ferramentas	`sysbench` 1.0.20, `fio` 3.41, `iperf3` 3.20	
Servidor de rede	`iperf3` 3.22 no Windows anfitrião	

IV. CONFIGURAÇÃO EXPERIMENTAL

A. Sistema Anfitrião e Máquinas Virtuais

Os dois cenários correram no mesmo portátil (Tabela I), com Windows 11 Home 25H2 (*build* 26200). A Tabela II resume as máquinas virtuais. Ambas têm os mesmos recursos e o mesmo sistema convidado, e confirmou-se que o *kernel* e as versões das ferramentas eram iguais nos dois cenários.

B. Instalação e Protocolo de Isolamento

O Hyper-V é uma funcionalidade opcional do Windows (`Microsoft-Hyper-V-All`), mas não aparece na edição Home. Foi instalado com um *script* que adiciona, com o DISM (*Deployment Image Servicing and Management*), os pacotes do Hyper-V que já vêm no Windows e depois ativa a funcionalidade, seguido de reinício [15]. A máquina foi criada no Gestor Hyper-V como Geração 2, sem memória dinâmica. Antes dos testes, o *checkpoint* automático foi fundido, para o disco não ficar como um VHDX de diferenciação. O VMware Workstation foi instalado com o instalador oficial, e a máquina foi criada com a configuração predefinida para Ubuntu de 64 bits. Nos dois casos, o Ubuntu Server 26.04 foi instalado com as opções predefinidas.

Por omissão, o Hyper-V mostra ao convidado os dois vCPU como dois *threads* SMT do mesmo núcleo, enquanto o VMware os mostra como processadores independentes. Para a comparação depender só do hipervisor, desligou-se o SMT no Hyper-V (`Set-VMProcessor -HwThreadCountPerCore 1`). Assim, os dois convidados ficaram com dois vCPU independentes, sem SMT, num só nó NUMA (*Non-Uniform Memory Access*), o que se confirmou com o `lscpu`. A única diferença que fica é a forma como os vCPU são agrupados: num *socket* no Hyper-V e em dois no VMware. O Hyper-V não deixa definir *sockets* sem criar nós NUMA virtuais, por isso esta diferença foi mantida. Não afeta o desempenho, porque o anfitrião só tem um processador físico.

Cada cenário precisou de um modo de arranque diferente do anfitrião, confirmado depois de cada reinício:

- **Hyper-V:** hipervisor ativo (`hypervisorlaunchtype auto`) e escalonador *root* (tipo `0x4`, visto no registo de eventos do Windows).
- **VMware:** hipervisor e Integridade de Memória desligados (`hypervisorlaunchtype off`), com `HypervisorPresent = False` e o ficheiro `vmware.log` a indicar `Monitor Mode: CPL0`.

Esta verificação é essencial: se o hipervisor do Windows estivesse ativo, o VMware correria em modo `ULM`, sobre o Hyper-V, e já não seria um Tipo 2.

Nos dois cenários: só uma máquina virtual ligada de cada vez; aplicações do anfitrião fechadas (incluindo a sincronização do OneDrive) e Windows Update em pausa; pastas das máquinas excluídas da análise em tempo real do Microsoft Defender; plano de energia "Elevado desempenho" e portátil sempre ligado à corrente. Antes de cada bateria de testes, esperou-se 10 minutos depois do arranque e suspenderam-se as atualizações automáticas do convidado (`apt`). Os testes foram lançados por SSH (*Secure Shell*), dentro de uma sessão `tmux` no convidado.

C. Cargas de Trabalho

Os testes foram automatizados num só *script*, igual nos dois convidados (Tabela III). Cada bateria começa com uma corrida de aquecimento, que é descartada, seguida de $N = 10$ repetições. Em cada repetição correm, por ordem, o processador, a memória e o disco, para que uma perturbação passageira do anfitrião não afete só um dos testes. Há pausas de 5 s entre testes, e a *cache* de páginas do convidado é limpa antes de cada teste de disco (`drop_caches`). Na rede, usou-se um servidor `iperf3` no Windows anfitrião, também com aquecimento e 10 repetições. Como a configuração é de nó único (*single node*), o tráfego vai do convidado ao anfitrião apenas pelo comutador virtual de cada hipervisor, sem passar por uma placa de rede física. Assim, mede-se só o custo da virtualização da rede, que de outra forma ficaria escondido pelo limite de uma ligação física (cerca de 1 Gbit/s), igual nos dois cenários. Os comandos, os *scripts* e os dados brutos estão no repositório do trabalho [15].

Sistema de referência. Para medir o *overhead* face ao sistema sem virtualização, os testes foram repetidos no mesmo portátil, sem nenhuma máquina virtual, com as mesmas ferramentas e parâmetros. No processador e na memória, o `sysbench` correu num Ubuntu 26.04.1 nativo, arrancado a partir de uma *pen* USB (*live*, *kernel* 7.0.0-30), com o perfil de energia "desempenho". Os dois *threads* foram fixados em dois núcleos físicos diferentes (`taskset`), tal como os dois vCPU sem SMT das máquinas virtuais. O *turbo* foi desligado (`intel_pstate/no_turbo`), o que limita o processador a 2,4 GHz. A razão é que, no Windows anfitrião, o processador corre sempre a cerca de 2,35 GHz, sem *turbo*, em todos os

Tabela III
CARGAS DE TRABALHO (POR REPETIÇÃO)

Subsistema	Ferramenta e parâmetros	Duração
CPU	`sysbench cpu`, primos até 20 000; 1 e 2 *threads*	10 s
Memória	`sysbench memory`, escrita seq., bloco 1 KiB	10 s
	idem, escrita sequencial, bloco 1 MiB	10 s
	idem, leitura sequencial, bloco 1 MiB	10 s
	idem, leitura aleatória num *buffer* de 256 MiB	10 s
Disco	`fio`, `libaio`, `direct=1`, ficheiro de 4 GiB: leitura e escrita aleatórias 4 KiB, $Q = 1$; mista 50/50 aleatória 4 KiB, $Q = 1$ e $Q = 32$; leitura e escrita sequenciais 1 MiB, $Q = 16$	3+15 s
Rede	`iperf3` TCP: convidado→anfitrião, inverso, 4 ligações	10 s
	`iperf3` UDP: 1 Mbit/s, 100 Mbit/s e 1 Gbit/s	10 s
	`ping`: 50 pacotes ICMP a 200 ms (RTT)	10 s

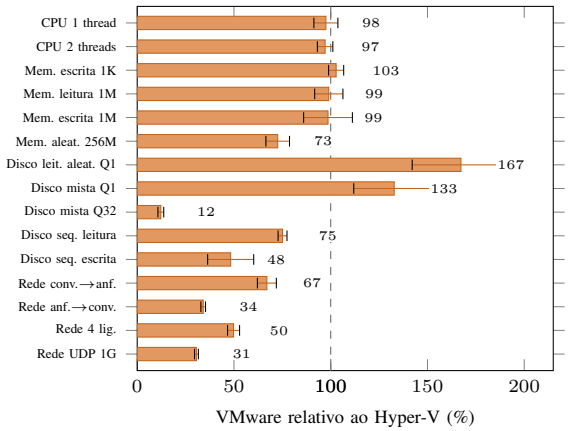


Figura 2. Débito do Tipo 2 relativo ao Tipo 1 (Hyper-V = 100%, tracejado). Barras de erro: $\sigma_{\text{VMware}}/\mu_{\text{Hyper-V}}$ (não incluem a dispersão do Hyper-V, indicada na Tabela IV).

planos de energia, e foi a essa frequência que correram as máquinas virtuais. Com o *turbo*, o nativo chega a cerca de 4 GHz e é 67–71% mais rápido no processador, o que mediria a frequência e não a virtualização. No disco e na rede, a referência foi o Windows anfitrião, com o hipervisor desligado (o mesmo modo do cenário VMware): o `fio` 3.41 com os seis perfis da Tabela III (`windowsaio`, `direct=1`, ficheiro de 4 GiB) e o `iperf3` 3.22 em *loopback* (127.0.0.1), que dá o limite máximo da pilha de rede do anfitrião. Em todos os casos, fizeram-se 10 repetições, com aquecimento.

V. RESULTADOS EXPERIMENTAIS

A Tabela IV junta as métricas principais, e a Fig. 2 mostra o desempenho do Tipo 2 em relação ao Tipo 1. Todas as repetições foram mantidas, incluindo três valores fora do normal no VMware (uma escrita sequencial de 266 MiB/s, uma corrida de CPU com 1 *thread* a 245 eventos/s e um RTT (*Round-Trip Time*) máximo de 10 ms). Estes valores aumentam o desvio-padrão, mas não mudam as conclusões.

A. Processador

Com um *thread*, o Hyper-V obteve 295,2 eventos/s e o VMware 287,9 ($-2{,}5\%$, $p = 0{,}241$). Com dois *threads*,

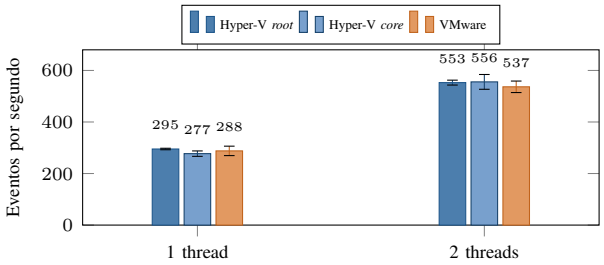


Figura 3. Processador ($\mu \pm \sigma$): paridade entre os três cenários com dois *threads*.

obtiveram 552,8 e 536,5 ($-2{,}9\%$, $p = 0{,}055$). Nenhuma das diferenças é significativa, e os dois escalam quase da mesma forma de 1 para 2 *threads* (1,87 no Hyper-V e 1,86 no VMware). Isto está de acordo com a literatura: com virtualização assistida por hardware, o código do convidado corre diretamente no processador nas duas arquiteturas [9], [13], e fica a menos de 5% do nativo com um *thread* (Tabela V). A diferença está na *estabilidade*. O coeficiente de variação foi de 1,0–1,7% no Hyper-V e de 4,2–6,4% no VMware. Uma explicação provável é que, no Tipo 2, os vCPU são *threads* de um processo normal e competem com as outras tarefas do Windows.

Efeito do escalonador. O teste de processador foi repetido com o *core scheduler* (`hypervisorschedulertype core`), o escalonador por omissão do Windows Server, com a mesma topologia (Fig. 3). Com dois *threads*, o resultado foi igual (555,6 contra 552,8 eventos/s, $+0{,}5\%$, $p = 0{,}77$). Com um *thread*, foi 6,1% mais baixo (277,2 contra 295,2, $p < 0{,}001$) e mais irregular (percentil 95 de 4,56 contra 3,59 ms). Ou seja, numa máquina com vCPU independentes, o escalonador não muda o débito total. A pequena perda com um *thread* pode dever-se ao trabalho extra que o *core scheduler* tem para manter cada núcleo físico isolado [11]. Assim, o escalonador do Windows 11 não dá vantagem ao Hyper-V em relação ao do Windows Server.

B. Memória

Nos três testes sequenciais, as diferenças vão de $-1{,}4\%$ a $+2{,}8\%$ e nenhuma é significativa: a largura de banda da RAM (blocos de 1 MiB, maiores do que as *caches*) e o acesso dentro da *cache* L1 (blocos de 1 KiB) são iguais nos dois. Já o acesso *aleatório* num *buffer* de 256 MiB é 27,4% mais lento no VMware (441 contra 608 MiB/s, $p < 0{,}001$). É o único teste de memória que provoca falhas de TLB constantes e, por isso, obriga a percorrer as tabelas de páginas do convidado e do hipervisor. Este custo depende do tamanho das páginas que o hipervisor usa para a memória do convidado [10]. A diferença é compatível com o Hyper-V usar páginas grandes, enquanto no VMware a memória do convidado pertence ao processo `vmware-vmx` e o Windows a atribui, por omissão, em páginas de 4 KiB. Em resumo, no acesso sequencial a memória não tem *overhead* relativo (embora os dois fiquem 12–20% abaixo do nativo), mas no Tipo 2 há um custo claro em cargas com acessos dispersos, como bases de dados.

Tabela IV
RESULTADOS: HYPER-V (TIPO 1, ESCALONADOR *root*) VS. VMWARE WORKSTATION (TIPO 2), $\mu \pm \sigma$, $N = 10$

Subsistema	Métrica	Hyper-V	VMware	$\Delta\%$	p (Welch)	Melhor
CPU	Eventos/s, 1 *thread*	295,2 ± 2,9	287,9 ± 18,3	−2,5	0,241	n.s.
	Eventos/s, 2 *threads*	552,8 ± 9,5	536,5 ± 22,3	−2,9	0,055	n.s.
	Latência p95, 2 *threads* (ms)	4,44 ± 0,06	4,58 ± 0,25	+3,0	0,132	n.s.
Memória (MiB/s)	Escrita seq., bloco 1 KiB	4 226 ± 267	4 346 ± 163	+2,8	0,244	n.s.
	Leitura seq., bloco 1 MiB	29 140 ± 859	28 840 ± 2 122	−1,0	0,686	n.s.
	Escrita seq., bloco 1 MiB	22 499 ± 1 181	22 194 ± 2 835	−1,4	0,759	n.s.
	Leitura aleatória, 256 MiB	**608 ± 33**	**441 ± 37**	**−27,4**	**<0,001**	**Tipo 1**
Disco	Leitura aleatória 4K, $Q = 1$ (IOPS)	3 986 ± 183	6 668 ± 1 006	+67,3	<0,001	Tipo 2
	latência média / p99 (μs)	249 / 356	151 / 182	−39,5 / −49,0	<0,001 / <0,001	Tipo 2
	Escrita aleatória 4K, $Q = 1$ (IOPS)	5 393 ± 295	6 282 ± 770	+16,5	0,006	Tipo 2
	Mista 4K, $Q = 1$ (IOPS totais)	4 656 ± 206	6 183 ± 971	+32,8	<0,001	Tipo 2
	Mista 4K, $Q = 32$ (IOPS totais)	**75 399 ± 2 698**	**9 161 ± 1 152**	**−87,9**	**<0,001**	**Tipo 1**
	Leitura sequencial 1M, $Q = 16$ (MiB/s)	1 293 ± 31	972 ± 30	−24,9	<0,001	Tipo 1
	Escrita sequencial 1M, $Q = 16$ (MiB/s)	1 651 ± 232	798 ± 196	−51,7	<0,001	Tipo 1
Rede	TCP convidado→anfitrião (Mbit/s)	3 546 ± 241	2 374 ± 172	−33,0	<0,001	Tipo 1
	TCP anfitrião→convidado (Mbit/s)	6 165 ± 539	2 100 ± 73	−65,9	<0,001	Tipo 1
	TCP 4 ligações (Mbit/s)	4 584 ± 767	2 282 ± 143	−50,2	<0,001	Tipo 1
	RTT médio / máximo (ms)	0,46 / 0,78	0,51 / 2,33	+9,6 / +197	0,287 / 0,129	n.s.
	UDP 1 Mbit/s: *jitter* (ms)	0,044 ± 0,027	0,008 ± 0,005	−82,7	0,002	Tipo 2
	UDP 100 Mbit/s: *jitter* (ms)	0,020 ± 0,010	0,126 ± 0,039	+547	<0,001	Tipo 1
	UDP 1 Gbit/s: débito (Mbit/s) / perda (%)	977 / 2,3	299 / 0,0	−69,4 / –	<0,001 / 0,003	Tipo 1

n.s.: diferença não significativa ($p \geq 0{,}05$). Convenção de sinais de $\Delta\%$ conforme a Secção III-C.

C. Armazenamento

No disco, o resultado depende do número de pedidos em fila.

Um pedido de cada vez ($Q = 1$): vantagem do Tipo 2. Com um pedido de cada vez, o VMware teve mais 67% de IOPS em leitura aleatória, com latência média de 151 μs contra 249 μs e um percentil 99 de 182 μs contra 356 μs. A explicação mais provável é a *cache* de ficheiros do Windows. O ficheiro de teste de 4 GiB cabe nos 16 GB do anfitrião, e o `vmware-vmx` lê o `.vmdk` através do sistema de ficheiros. Assim, as leituras "diretas" do convidado acabam por ser servidas pela RAM do anfitrião: a opção `direct=1` evita a *cache* do convidado, mas não a do anfitrião. A escrita beneficia do mesmo efeito (+16,5% IOPS), porque o convidado recebe a confirmação antes de os dados chegarem ao SSD. Isto melhora a latência, mas põe os dados em risco se houver uma falha de energia. No Hyper-V, cada pedido vai pelo VMBus até ao NVMe, sem esta *cache*.

Vários pedidos em fila: vantagem do Tipo 1. Com 32 pedidos em fila, o Hyper-V passou de 4 656 para 75 399 IOPS (16 vezes mais), com uma latência média (entre leituras e escritas) de apenas 424 μs. O VMware passou de 6 183 para 9 161 (1,5 vezes), com uma latência de 3 541 μs, ou seja, −88% ($p < 0{,}001$). A lei de Little (3) confirma os dois valores: $32/424\,\mu\text{s} \approx 75\,500$ e $32/3\,541\,\mu\text{s} \approx 9\,036$ IOPS, a menos de 1,5% do medido. No VMware, a latência cresce na proporção da fila e o débito fica preso em cerca de 9 000 IOPS. O caminho de E/S está, portanto, *serializado*, o que é de esperar num controlador emulado (LSI Logic) tratado por um processo em modo utilizador. O controlador do Hyper-V, pelo contrário, envia os pedidos em paralelo para o NVMe. O mesmo limite vê-se no débito sequencial com blocos de 1 MiB: −25% em leitura e −52% em escrita no Tipo 2 (Fig. 4).

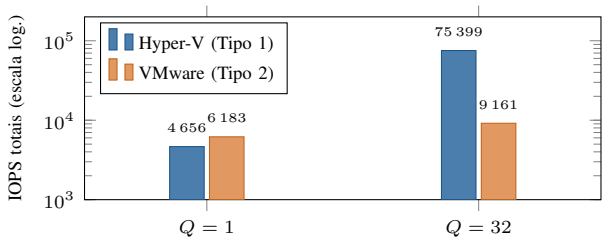


Figura 4. E/S aleatória mista 4 KiB: o Tipo 2 é mais rápido com $Q = 1$ (*cache* do anfitrião), mas não escala com a profundidade de fila.

D. Rede

Em TCP (*Transmission Control Protocol*), o Hyper-V foi 33% mais rápido do convidado para o anfitrião, 66% mais rápido no sentido inverso e 50% mais rápido com quatro ligações em paralelo, sempre com $p < 0{,}001$ (Fig. 5). No sentido inverso, o `iperf3` do anfitrião chegou a 87% de CPU, por isso o valor do Hyper-V pode estar limitado pelo próprio gerador de tráfego. A latência ida-e-volta média é igual (0,46 contra 0,51 ms, $p = 0{,}29$), mas o VMware tem picos até 2,3 ms e uma variação quatro vezes maior. A causa é o tipo de placa de rede. A `e1000` do VMware é emulada: cada acesso aos seus registos obriga a sair para o monitor e a processar no `vmware-vmx`. A `hv_netvsc` do Hyper-V troca pacotes em lote pelo VMBus. Ao enviar a partir do convidado, o `iperf3` do anfitrião gastou 31,6% de CPU no VMware e 9,4% no Hyper-V, com um débito mais baixo: cerca de 75 contra 376 Mbit/s por ponto percentual de CPU. No VMware, cada bit recebido custa mais CPU ao anfitrião.

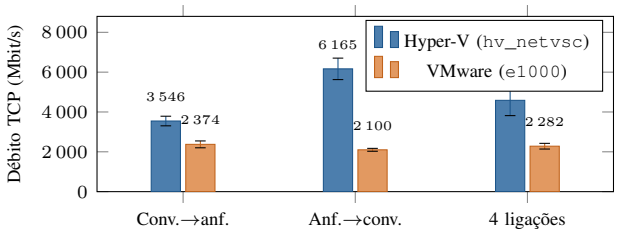


Figura 5. Débito de rede convidado↔anfitrião ($\mu \pm \sigma$).

Com a placa paravirtualizada `vmxnet3`, a diferença seria provavelmente menor, mas não foi testada, para manter a configuração predefinida (Secção VI). Os testes UDP (*User Datagram Protocol*) mostram o mesmo. Com pouco tráfego (1 Mbit/s), o *jitter* é muito baixo nos dois (0,044 contra 0,008 ms). Com 100 Mbit/s, o *jitter* do VMware é seis vezes maior (0,126 contra 0,020 ms, $p < 0{,}001$). A 1 Gbit/s, o Hyper-V entrega 977 Mbit/s com 2,3% de perda, e o VMware não passa dos 299 Mbit/s. O VMware não perde pacotes só porque não consegue enviar mais: o limite está na própria placa emulada, que envia cerca de um terço dos pacotes UDP do Tipo 1.

Tal como Louro [3], usou-se o `iperf3` em TCP, para o débito, e em UDP, para o *jitter*. No trabalho de Louro, porém, o tráfego passava pela rede física entre os dois nós de um *cluster*, com uma janela TCP fixa de 20 KB. As máquinas sobre Hyper-V chegaram a 207–222 Mbit/s e a um *jitter* de 0,39–1,07 ms [3, Figs. 4.3–4.4]. Como os dois trabalhos medem caminhos diferentes, os valores absolutos não são comparáveis.

E. Overhead Face ao Sistema Nativo

A Tabela V compara cada hipervisor com o sistema de referência, usando a Eq. (2) com o nativo no lugar do Hyper-V. No processador, o *overhead* é pequeno nos dois: $-1{,}9\%$ (Hyper-V) e $-4{,}4\%$ (VMware) com um *thread*, e $-8{,}2\%$ e $-10{,}9\%$ com dois. O nativo escala perfeitamente de 1 para 2 *threads* (fator 2,00), contra 1,87 e 1,86 nas máquinas virtuais, e tem uma latência p95 de 3,30 ms, contra 4,44 e 4,58 ms. Assim, o custo de virtualizar o processador está sobretudo em correr os dois vCPU ao mesmo tempo, em competição com os *threads* do anfitrião, e não em cada instrução. Cerca de 2 pontos percentuais vêm da diferença de frequência (2,35 contra 2,40 GHz).

Na memória sequencial com blocos de 1 MiB, os dois hipervisores perdem 12–20% face ao nativo, sem diferença significativa entre eles. No acesso aleatório, o *overhead* é de 26,8% no Hyper-V e de 46,9% no VMware. É o maior custo no processador e na memória, o que confirma que a tradução de endereços em dois níveis é o principal custo da virtualização da memória, e que é maior no Tipo 2 (Secção V-B). O teste com blocos de 1 KiB foi *mais rápido* nas máquinas virtuais (+16 a +20%). Neste teste, cada bloco conta como um evento, e o resultado depende do custo fixo que o `sysbench` tem por evento, que muda de ambiente para ambiente. Por isso, não é visto como um ganho da virtualização.

Tabela V
OVERHEAD FACE AO SISTEMA NATIVO (μ, $N = 10$)

Métrica	**Nativo**	**Hyper-V**	**VMware**
Linux nativo, 2,4 GHz sem turbo (sysbench)			
CPU, 1 *thread* (eventos/s)	301,0	$-1{,}9\%$	$-4{,}4\%$
CPU, 2 *threads* (eventos/s)	602,1	$-8{,}2\%$	$-10{,}9\%$
CPU, latência p95, 2 *threads* (ms)	3,30	$+34{,}6\%$	$+38{,}7\%$
Mem. escrita seq. 1 KiB (MiB/s)	3 633	$+16{,}3\%$	$+19{,}6\%$
Mem. leitura seq. 1 MiB (MiB/s)	33 199	$-12{,}2\%$	$-13{,}1\%$
Mem. escrita seq. 1 MiB (MiB/s)	27 882	$-19{,}3\%$	$-20{,}4\%$
Mem. leitura aleat. 256 MiB (MiB/s)	**831**	$\mathbf{-26{,}8\%}$	$\mathbf{-46{,}9\%}$
Windows anfitrião, hipervisor desligado (fio, windowsaio)			
Leitura aleat. 4K, $Q = 1$ (IOPS)	7 758	$-48{,}6\%$	$-14{,}1\%$
latência média (μs)	127	$+95{,}5\%$	$+18{,}3\%$
Escrita aleat. 4K, $Q = 1$ (IOPS)	15 382	$-64{,}9\%$	$-59{,}2\%$
Mista 4K, $Q = 1$ (IOPS totais)	10 259	$-54{,}6\%$	$-39{,}7\%$
Mista 4K, $Q = 32$ (IOPS totais)	54 494	$+38{,}4\%$	$-83{,}2\%$
Leitura seq. 1M, $Q = 16$ (MiB/s)	1 218	$+6{,}2\%$	$-20{,}2\%$
Escrita seq. 1M, $Q = 16$ (MiB/s)	1 910	$-13{,}5\%$	$-58{,}2\%$
Windows anfitrião, iperf3 em loopback (indicativo)			
TCP envio/inverso (Mbit/s)	18 084/17 614	$-80/-65\%$	$-87/-88\%$
TCP 4 ligações (Mbit/s)	45 070	$-89{,}8\%$	$-94{,}9\%$

Todas as diferenças face ao nativo são significativas ($p < 0{,}05$, Welch). Desvios-padrão e intervalos de confiança a 95% no ficheiro de resultados [15].

No disco, a referência é o Windows anfitrião, que usa outra pilha de E/S (NTFS e `windowsaio`, em vez de `ext4` e `libaio`), por isso a comparação serve apenas de indicação. Com $Q = 1$, os dois hipervisores perdem 14–65% de IOPS face ao anfitrião, e a latência média de leitura do Hyper-V é quase o dobro da nativa (249 contra 127 μs). É o custo, em cada pedido, de passar pelo VMBus e pela partição raiz. O VMware perde menos em leitura (-14%) porque parte das leituras vem da *cache* do anfitrião. Com $Q = 32$ e em leitura sequencial, o Hyper-V fica *acima* da referência ($+38\%$ e $+6\%$). Não é a virtualização que acelera o disco: o `libaio` do Linux aguenta mais pedidos em simultâneo do que o `windowsaio` do `fio` no Windows. O VMware, pelo contrário, fica a 17% do anfitrião com $Q = 32$ e a 42% em escrita sequencial. Na rede, o *loopback* do anfitrião chegou a 18 084 Mbit/s em TCP, 17 614 no sentido inverso e 45 070 com quatro ligações. As máquinas virtuais chegam a 10–35% deste limite no Hyper-V e a 5–13% no VMware. Como o *loopback* não passa por nenhum dispositivo de rede, estes valores mostram apenas a margem que cada comutador virtual deixa por aproveitar.

VI. LIMITAÇÕES E AMEAÇAS À VALIDADE

- **Referência:** não há um único sistema nativo equivalente às duas máquinas virtuais. O processador e a memória foram comparados com um Linux nativo arrancado por USB (*kernel* 7.0.0-30, contra 7.0.0-38 nas máquinas virtuais), a 2,4 GHz fixos. O disco e a rede foram comparados com o Windows anfitrião (NTFS e `windowsaio`; e *loopback*, porque a ligação convidado↔anfitrião não existe sem virtualização). Por isso, no disco e na rede, o *overhead* face ao nativo inclui as diferenças entre sistemas operativos, e não só o custo do hipervisor.

- **Dispositivos predefinidos:** o VMware usou dispositivos emulados (LSI Logic, `e1000`) e o Hyper-V paravirtualizados. É a configuração por omissão de cada produto, mas aumenta a diferença em relação a um VMware com `vmxnet3` e PVSCSI (SCSI paravirtualizado) ou NVMe. Os escalonadores de E/S do convidado também foram diferentes (`none` e `mq-deadline`).
- ***Cache* do anfitrião no Tipo 2:** o ficheiro de teste de 4 GiB cabe na memória do anfitrião, e o `direct=1` só evita a *cache* do convidado. Assim, com $Q = 1$, o VMware é medido em parte a partir da RAM do anfitrião e o Hyper-V a partir do NVMe. Os dois caminhos não são equivalentes, e a vantagem do Tipo 2 com $Q = 1$ não quer dizer que o seu disco virtual seja mais rápido.
- **Modo do anfitrião:** no cenário Hyper-V, o Windows correu como partição raiz, com a segurança baseada em virtualização ativa; no cenário VMware, correu sem hipervisor. Esta diferença é inevitável quando as duas arquiteturas estão no mesmo equipamento.
- **Causa da diferença na memória:** a explicação para a leitura aleatória (tamanho das páginas) baseia-se no padrão dos resultados e na literatura, mas não foi medida diretamente.
- **Equipamento:** um portátil com Windows 11 Home (edição em que o Hyper-V não é suportado oficialmente), em nó único e com 2 vCPU e 4 GB por máquina. Os resultados podem não se aplicar a servidores nem a infraestruturas com vários nós, onde a rede física também conta.
- ***Benchmarks* sintéticos:** medem subsistemas isolados e não cargas reais, que os combinam.

VII. Conclusões

Este trabalho comparou um hipervisor nativo (Hyper-V) e um hipervisor *host-based* (VMware Workstation 17.6) no mesmo equipamento, com máquinas virtuais Ubuntu Server 26.04 com os mesmos recursos e vCPU independentes. Nas condições testadas:

1) **O processador e a memória sequencial** não têm diferenças significativas entre os dois hipervisores, graças à virtualização assistida por hardware. O Hyper-V foi, no entanto, mais estável (coeficiente de variação abaixo de 2%).
2) **O acesso aleatório à memória** é 27% mais lento no Tipo 2, o que é compatível com um maior custo da tradução de endereços em dois níveis.
3) **No disco**, o Tipo 2 é mais rápido com pedidos isolados, porque a *cache* do anfitrião serve as leituras e confirma as escritas antes de chegarem ao SSD. Com vários pedidos em simultâneo, porém, fica limitado a cerca de 9 000 IOPS, como a lei de Little confirma, e perde 88% de IOPS com $Q = 32$ e 25–52% de débito sequencial.
4) **Na rede**, o Tipo 1 é 33–66% mais rápido em TCP, consegue três vezes mais débito em UDP e tem seis vezes menos *jitter* com carga moderada, graças aos dispositivos paravirtualizados.
5) **Face ao sistema nativo**, à mesma frequência, o processador perde 2–11% nos dois hipervisores, sobretudo quando os dois vCPU trabalham ao mesmo tempo, e a memória sequencial perde 12–20%. O maior custo está no acesso aleatório à memória: 27% no Tipo 1 e 47% no Tipo 2.
6) **O escalonador do Hyper-V** (*root* ou *core*) não muda o débito de uma máquina com vCPU independentes.

Para cargas típicas de centro de dados, com muita E/S em simultâneo, rede intensiva e acessos dispersos à memória, os resultados favorecem claramente o hipervisor nativo. O Tipo 2 continua a ser adequado para computadores de trabalho, com cargas interativas e pouco paralelas. Num anfitrião Windows, é também essencial confirmar que o hipervisor de Tipo 2 corre no seu modo nativo (`CPL0`) e não sobre o Hyper-V. Como trabalho futuro, propõe-se repetir os testes com dispositivos paravirtualizados no VMware e avaliar cargas reais, como bases de dados e servidores web.

Referências

[1] Microsoft, "Hyper-V virtualização no Windows Server e no Windows," [Online]. Available: https://learn.microsoft.com/pt-pt/windows-server/virtualization/hyper-v/overview. [Accessed: Oct. 18, 2026].

[2] Broadcom, "VMware Workstation Pro 17.0 documentation," [Online]. Available: https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/workstation-pro/17-0.html. [Accessed: Oct. 25, 2026].

[3] A. F. C. Louro, "Comparação do desempenho de infraestruturas virtualizadas de elevada disponibilidade usando hypervisors nativos ou containers," M.Sc. dissertation, Dept. Eng. Informática, Univ. Beira Interior, Covilhã, Portugal, 2018. [Online]. Available: https://ubibliorum.ubi.pt/handle/10400.6/9837.

[4] Universidade da Beira Interior, "Conteúdos das aulas teóricas da unidade curricular Virtualização e Centros de Dados," [Online]. Available: https://moodle.ubi.pt/. [Accessed: Oct. 18, 2026].

[5] A. Kopytov, "sysbench: Scriptable database and system performance benchmark," GitHub, 2020. [Online]. Available: https://github.com/akopytov/sysbench.

[6] J. Axboe, "Flexible I/O Tester (fio)," GitHub. [Online]. Available: https://github.com/axboe/fio. [Accessed: Oct. 18, 2026].

[7] G. J. Popek and R. P. Goldberg, "Formal requirements for virtualizable third generation architectures," *Commun. ACM*, vol. 17, no. 7, pp. 412–421, 1974.

[8] P. Barham *et al.*, "Xen and the art of virtualization," in *Proc. 19th ACM Symp. Operating Systems Principles (SOSP)*, 2003, pp. 164–177.

[9] K. Adams and O. Agesen, "A comparison of software and hardware techniques for x86 virtualization," in *Proc. 12th Int. Conf. ASPLOS*, 2006, pp. 2–13.

[10] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, "Accelerating two-dimensional page walks for virtualized systems," in *Proc. 13th Int. Conf. ASPLOS*, 2008, pp. 26–35.

[11] Microsoft, "Manage Hyper-V hypervisor scheduler types," [Online]. Available: https://learn.microsoft.com/windows-server/virtualization/hyper-v/manage/manage-hyper-v-scheduler-types. [Accessed: Oct. 22, 2026].

[12] Microsoft, "Windows Hypervisor Platform," [Online]. Available: https://learn.microsoft.com/virtualization/api/. [Accessed: Oct. 22, 2026].

[13] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE ISPASS*, 2015, pp. 171–172.

[14] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[15] H. Laia, A. Santos e D. Basílio, "Hyper-V vs. VMware: comandos, *scripts* e dados experimentais," GitHub, 2026. [Online]. Available: https://github.com/henriquelaia/vcd-hyperv-vs-vmware.